A SAFETY-ORIENTED EXECUTION ALGORITHM SERVICE FOR BINANCE USD-M BTCUSDT

CANDIDATE PROJECT REPORT

Haoran Wu
Calais Quantitative Development Candidate Project
`calais-execution-algorithm`

June 2026

ABSTRACT

This report describes a compact execution algorithm service for Binance USD-M Futures BTCUSDT Perpetual. The service accepts a final target position, a permitted active execution price range, a target duration, and either a Chase or TWAP execution mode. The design priority is small-but-correct execution behavior rather than broad venue coverage: one symbol, one exchange family, one-way mode, deterministic simulator coverage, and a credential-gated Binance Testnet path. The core safety argument is an exposure invariant that accounts for confirmed fills, live open quantity, pending submits, pending cancels, unknown create outcomes, and proposed child quantity before any new order can be submitted. Deterministic simulator runs cover races that Testnet cannot reliably force, including fill-after-cancel and ambiguous create-timeout reconciliation. Accepted sanitized Binance Testnet artifacts are included for both Chase and TWAP; they preserve order/trade evidence while redacting account balance and position updates. The report itself uses the requested arxiv-style template [Kour, 2026].

Keywords execution algorithm · order state machine · overfill prevention · deterministic simulator · TWAP · Chase order

1 Introduction

The project brief asks for a Python 3.11+ execution service for Binance USD-M Futures BTCUSDT Perpetual [Calais, 2026]. The service receives a final account target position, a price interval, a target duration, and an execution algorithm. The most important interpretation is that `target_position` is not an order quantity. The system first reads the current position and computes:

$$required\_trade\_quantity = target\_position - current\_position. \tag{1}$$

A negative-to-positive target therefore requires trading through zero. For example, moving from -0.003 BTC to $+0.005$ BTC requires buying 0.008 BTC, not 0.005 BTC. This rule is intentionally placed at execution creation, before algorithm selection, so Chase and TWAP share the same target-position semantics.

The second important interpretation is that price, time, and quantity cannot all be guaranteed across every market path. If a buy price range ends at an upper bound, the implementation cannot legally force a fill above that bound. If the market never becomes executable within the interval, a correct result may be partial or fully unfilled. The report therefore evaluates the project by whether the system reports actual outcome and preserves safety, not whether every run reaches 100 percent completion.

The implementation follows a "small and correct" strategy. It avoids multi-asset routing, durable production recovery, and mainnet automation. In exchange, the core order lifecycle is explicit, deterministic tests cover abnormal event ordering, and every child submit passes through the same exposure gate.

Table 1: How the report maps to the visible scoring categories.

Scoring area	Report emphasis
Execution correctness	Target-position delta, Decimal math, price bounds, exposure invariant, create-timeout handling.
Edge cases and state	Parent and child state machines, cancel/fill race, duplicate events, stale streams, deadline behavior.
Testing quality	Deterministic simulator, T1-T10 matrix, raw artifacts, repeatable verification commands.
Algorithm design	Chase repricing policy and TWAP absolute schedule with carry-forward deficit.
Architecture and communication	Clear ownership boundaries, limitations, AI disclosure, and sanitized Testnet evidence status.

Table 2: Compact requirement coverage. More detailed edge-case and test mappings are in the appendix.

Requirement area	Status	Evidence or note
Target-position semantics	COVERED	Engine computes net delta from current position; no-action and cross-zero cases are tested.
Chase and TWAP	COVERED	Chase passive repricing and TWAP absolute schedule are implemented as narrow algorithm helpers.
Price bounds and Decimal math	COVERED	Decimal-string API fields, Decimal internal math, side-aware rounding, tick/step/min checks.
State machines and overfill safety	COVERED	Parent and child states are monotonic; submit path enforces the exposure invariant.
Deterministic simulator	COVERED	Simulator covers partial fill, cancel/fill race, post-only rejection, create timeout, duplicate/lower fills, stale data, and stream disconnect.
Binance Testnet integration code	COVERED	Adapter and runner implement symbol rules, REST mutation, exact lookup, user streams, and evidence artifacts.
Accepted Binance Testnet Chase/TWAP evidence	COVERED	Accepted sanitized Chase and TWAP artifacts are included with non-empty exchange order IDs.
Final reporting and AI disclosure	COVERED	This LaTeX report and `AI_USAGE.md` document decisions, evidence, and verification.

The scoring rubric makes this emphasis natural. Execution correctness and edge cases dominate the score, while report writing is a smaller but important communication channel. The report is therefore structured as a correctness defense. It starts with requirements and ownership boundaries, then spends most of its space on state, safety invariants, failure cases, and evidence. The goal is to make the implementation easy to challenge in an interview: each important claim should point to a module boundary, invariant, test, artifact, or limitation.

2 Architecture and Ownership Boundaries

The architecture is organized around one rule: the engine owns trading correctness. API and runtime code supervise execution, algorithms compute narrow decisions, and exchange adapters normalize venue behavior. They do not independently mutate exposure.

`ExecutionRuntime` is the API-facing supervisor. It builds the correct environment, rejects concurrent active executions for the same environment and symbol, advances nonterminal executions, supervises market and private streams for Binance-like adapters, renews listen keys, and reconciles after stream events or disconnects.

`ExecutionService` is deliberately thin. It delegates execution lifecycle to `ExecutionEngine`, which owns parent execution state, child order state, exposure buckets, fill accounting, cancel flow, create-timeout handling, deadline behavior, and terminal summaries. This boundary keeps correctness checks in one place.

`Chase` and `TWAP` helpers do not submit orders. Chase computes the passive desired price and whether repricing is due. TWAP computes schedule progress and deficit. Both route through the engine's submit path, so price checks, exposure checks, Decimal normalization, and unknown-order reservations remain common.

`ExchangeAdapter` gives the simulator and Binance adapter the same shape: symbol rules, position, best bid/ask, submit, cancel, exact order lookup, stream events, reconciliation, and stream health. This is important because deterministic simulator tests exercise the same engine logic used by the Binance Testnet runner.

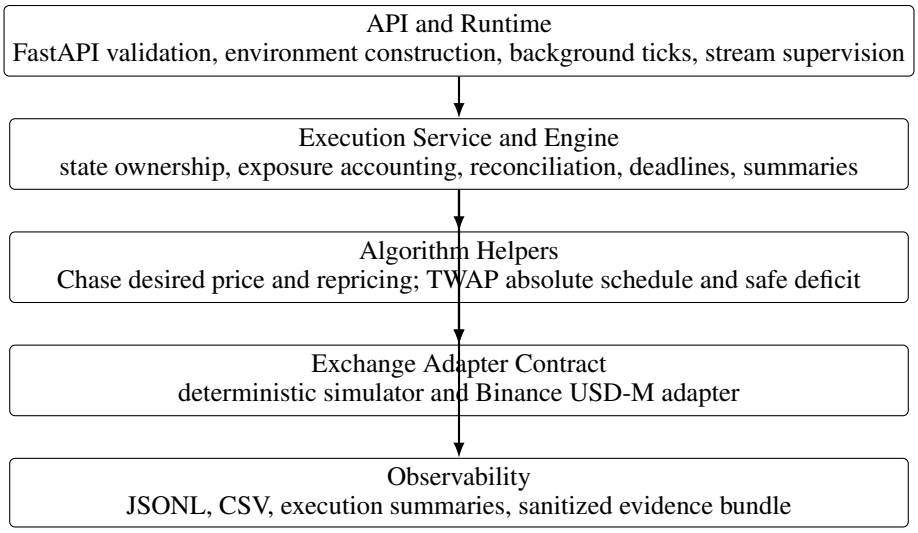

Figure 1: Layered ownership. The algorithm layer proposes; the engine decides whether the proposed child order is safe.

Phase	Allowed next phase	Purpose
CREATED	VALIDATING	Request accepted and domain objects built.
VALIDATING	RUNNING or terminal	Position, symbol rules, bounds, duration, and dust checks.
RUNNING	CANCELLING or terminal	Submit, cancel, reconcile, deadline, and fill processing.
CANCELLING	terminal	Stop active exposure, reconcile, then summarize.
Terminal	none	COMPLETED, PARTIALLY COMPLETED, EXPIRED, CANCELLED, or FAILED.

Figure 2: Parent execution state transition table. Terminal states are one-way.

3 Lifecycle and State Machines

The parent execution lifecycle is monotonic. Terminal executions do not return to RUNNING. The engine serializes operations per execution so API requests, background ticks, stream events, manual cancel, and reconciliation cannot concurrently mutate the same execution record.

The child state machine is designed for disagreement between REST responses and user-stream events. A fill can arrive after a cancel request. A create request can time out while the exchange still accepts the order. A REST snapshot can be older than a private stream event. The engine therefore treats cumulative fills as monotonic, keeps pending-cancel and unknown-create exposure reserved, and uses exact client-order lookup to repair ambiguous create outcomes.

4 Core Safety Invariant

The central correctness claim is the submit gate:

$$\begin{aligned} &confirmed\_filled + live\_open + pending\_submit \\ &+ pending\_cancel + unknown\_order + new\_child\_quantity \\ &\leq normalized\_target\_trade\_quantity. \end{aligned} \tag{2}$$

This is stronger than recomputing a simple remaining quantity. `confirmed_filled` is parent-level cumulative filled quantity. `live_open` can still trade at the venue. `pending_submit` protects the interval between local intent and exchange response. `pending_cancel` protects the period where a cancel request has been sent but fills can still arrive. `unknown_order` protects ambiguous create outcomes. `new_child_quantity` is the proposed order.

The two most important buckets are `pending_cancel` and `unknown_order`. If the system releases `pending_cancel` too early, it can submit a replacement while the old order is still fillable. If it releases `unknown_order` after a create

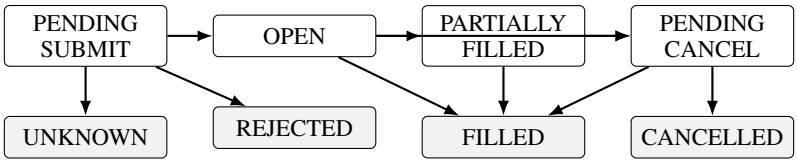

Figure 3: Child order states. UNKNOWN is a reserved-exposure state, not a harmless error.

timeout, it can retry with a fresh client order ID even though the original order may be live. Both cases can create predictable overfill. Equation 2 prevents this by reserving exposure until reconciliation proves it is gone.

Other invariants support this gate:

- Parent confirmed filled quantity is monotonic.
- Per-child cumulative filled quantity is monotonic.
- Duplicate trade or event messages cannot increase parent fill twice.
- A timed-out create remains UNKNOWN until exact `origClientOrderId` reconciliation.
- No aggressive child order may violate the configured price bound.
- Terminal execution state cannot return to RUNNING.

5 Chase Design

Chase uses passive limit orders at the current best queue price: buy orders at best bid and sell orders at best ask. Passive orders are post-only when the venue supports it. Repricing requires both a price movement threshold and a minimum interval:

$$reprice\_difference\_bps = \left| \frac{desired\_price}{active\_order\_price} - 1 \right| \times 10000. \tag{3}$$

The default repricing mode is `ADVERSE_ONLY`. A buy order reprices upward when the best bid moves up enough; a sell order reprices downward when the best ask moves down enough. This default is deliberately conservative. It avoids unnecessary cancels when the existing order is already passive at a more favorable price and reduces cancel storm risk. `TWO_SIDED` is still available for experiments or interview discussion, but the safer default is to avoid churn unless the market moves away from the resting order.

Partial fills are handled at the parent level. If an order partially fills and then a reprice is needed, replacement quantity is not the old child's original quantity and not merely the venue-reported remaining quantity. It is the target quantity minus parent confirmed fills minus all still-reserved exposure. This is the same submit path as Equation 2, so cancel/fill race behavior is handled by the engine rather than by Chase-specific logic.

Deadline behavior is explicit. Under `CANCEL_REMAINDER`, active orders are cancelled and actual unfilled quantity is reported. Under `AGGRESSIVE_WITHIN_RANGE`, the engine may attempt a bounded marketable limit: buy no higher than the upper bound and sell no lower than the lower bound. If the market is outside range, the correct result is partial or unfilled, not fake completion.

6 TWAP Design

TWAP is implemented as an absolute schedule, not a sleep loop. At monotonic elapsed time t:

$$scheduled\_cumulative\_quantity(t) = total\_trade\_quantity \times \frac{elapsed\_time}{total\_duration}. \tag{4}$$

$$quantity\_deficit(t) = scheduled\_cumulative\_quantity(t) - confirmed\_cumulative\_filled\_quantity(t). \tag{5}$$

Before submitting, the engine subtracts reserved exposure:

$$safe\_child\_quantity = quantity\_deficit - (live\_open + pending\_submit + pending\_cancel + unknown\_order). \quad (6)$$

This design solves the problems that make the invalid `slice_qty / sleep` implementation unsafe. Absolute schedule time avoids accumulated sleep drift. Unfilled earlier quantity naturally carries forward into the next deficit. Open or unknown earlier slices cannot be submitted twice because reserved exposure is subtracted. The final slice can absorb legal step-size rounding remainder, but the invariant still prevents exceeding the normalized target.

The deterministic TWAP artifact at `reports/evidence/simulation/twap/exec_61fadac604f4440a` shows the same schedule behavior. Over a 100 second target, it submitted five 20-second slices of 0.002 BTC each, reconciled every slice fill, and completed exactly 0.010 BTC without exceeding the scheduled deficit.

7 Precision, Risk, and Binance Integration

All trading prices and quantities use `Decimal`. API trading fields are decimal strings, and Binance parameters are serialized as strings. This matters because direct Python `float` construction can introduce non-exchange-valid prices or quantities before tick and step rounding.

Risk validation is side-aware. Passive buys must not round through the best ask or exceed the upper bound. Passive sells must not round through the best bid or go below the lower bound. Aggressive buy deadline orders cannot exceed the upper bound, and aggressive sell deadline orders cannot go below the lower bound. Quantity normalization floors to step size rather than rounding up into over-trading; dust is recorded or rejected according to the implemented rule.

The Binance adapter reads dynamic BTCUSDT symbol rules from exchange metadata, including tick size, quantity step, minimum quantity, minimum notional, trading status, and supported time-in-force assumptions [Binance, 2026]. Market data is based on best bid/ask updates. Private user-data events and REST reconciliation repair order and fill state. REST is also used for initialization, create/cancel mutation, exact order lookup, and reconciliation after disconnects.

Create and cancel mutations are classified conservatively. HTTP 408 and transport timeout on create map to an unknown create outcome; the child remains UNKNOWN and its quantity remains reserved. HTTP 408 and transport timeout on cancel keep the child in pending-cancel exposure until reconciliation. Mainnet mutation is hard disabled by default and requires explicit configuration; it is not part of the take-home demonstration path.

Secrets are runtime-only. API keys, API secrets, request signatures, listen keys, and signed payloads are sanitized from artifacts and are not committed.

8 Testing and Evidence

The deterministic simulator is mandatory because Testnet cannot reliably produce all failure modes. It can script market data, partial fills, fill during cancel, cancel delay, post-only rejection, create timeout where the order actually exists, duplicate or lower cumulative fill snapshots, stale market data, stream disconnects, and reconciliation snapshots. These tests are repeatable and run without credentials.

Current reproducible non-live baseline verification passed after the final evidence cleanup plan:

```
uv run pytest -q tests/unit tests/simulation  →  506 passed
```

Credentialed/network-enabled integration verification is separate: `uv run pytest -q tests/integration`. Those results should be reported only for a full credentialed local review where the Binance integration tests run successfully.

The four simulator scripts also ran successfully during this report build and wrote committed structured artifacts:

- `reports/evidence/simulation/chase/exec_c8dc942476764355`
- `reports/evidence/simulation/twap/exec_61fadac604f4440a`
- `reports/evidence/simulation/cancel-race/exec_669d47a536a94682`
- `reports/evidence/simulation/create-timeout/exec_3ddb8a47995348d0`

The repository contains Binance Testnet runner scripts for Chase and TWAP. They require credentials and an explicit `-confirm-send-orders` flag and never fall back to simulation. Accepted sanitized Testnet artifacts are included at `reports/evidence/testnet/chase/exec_3168600ee25b4193` and

Table 3: Required scenario coverage. File names identify the main evidence source; many scenarios also have unit-level regressions.

ID	Scenario	Evidence
T1	Normal Chase	`scripts/run_sim_chase.py`; engine and simulator order tests.
T2	Chase Reprice	Chase threshold/min-interval tests; summary metric coverage.
T3	Partial Fill + Cancel Race	`scripts/run_sim_cancel_race.py`; cancel-race simulator tests.
T4	Create Timeout	`scripts/run_sim_create_timeout.py`; exact lookup and unknown exposure tests.
T5	TWAP Carry-forward	TWAP absolute schedule and deficit tests.
T6	Tail Quantity	Decimal, dust, min-quantity, and step-size tests.
T7	Price Outside Range	Price-bound and deadline/outside-range tests.
T8	Stream Disconnect	Runtime stream restart, reconcile, and stale-market tests.
T9	Duplicate Event	Duplicate trade and cumulative-fill monotonicity tests.
T10	Cross-zero Position	Required-trade and account-position tests.

Table 4: Selected simulator results from current verification.

Run	Result
Normal Chase	`chase/exec_c8dc942476764355`, child `ce_c8dc94247676_1`: one passive buy child for 0.010 BTC at 50000.00, then a simulator fill and reconciliation to COMPLETED.
TWAP schedule	`twap/exec_61fadac604f4440a`: five 0.002 BTC slices over 100 seconds filled and reconciled to 0.010 BTC total with `overfill_quantity=0`.
Cancel/fill race	`cancel-race/exec_669d47a536a94682`: a 0.004 BTC fill arrived during cancel; replacement was 0.006 BTC, preserving total exposure at 0.010.
Create timeout	`create-timeout/exec_3ddb8a47995348d0`, child `ce_3ddb8a479953_1`: unknown exposure was 0.010 before exact reconciliation and 0 after the original order was found open; no fresh client order ID was used.

`reports/evidence/testnet/twap/exec_85bef3985ea3431a`. The Chase manifest records exchange order ID 16277695886. The TWAP manifest records exchange order IDs 16277882286 and 16277899689. These accepted artifacts preserve order/trade evidence while redacting `ACCOUNT_UPDATE` balance and position fields.

The older credentialed raw Testnet artifacts `reports/evidence/testnet/chase/exec_85051310eb714ebe` and `reports/evidence/testnet/twap/exec_30ed2b4cac4346a1` are retained only as rejected connectivity/error artifacts. Both reached Binance and failed before order acceptance with `BINANCE_-2019:Margin is insufficient.`; both manifests have `accepted_exchange_order_evidence=false` and empty `exchange_order_ids`.

9 Results and Metrics

Each execution summary is intended to reconstruct quantity, price, order, time, and safety behavior. Table 4 shows selected deterministic run outputs from the committed evidence bundle under `reports/evidence/simulation`.

The cancel/fill race artifact is the clearest overfill evidence. The initial child was 0.010 BTC. During cancel, 0.004 BTC filled. The replacement child was only 0.006 BTC. The final summary showed:

Metric	Value
required quantity	0.010 BTC
confirmed filled	0.004 BTC
live open	0.006 BTC
reserved exposure	0.006 BTC
unknown order	0 BTC
client order IDs	`ce_669d47a536a9_1`, `ce_669d47a536a9_2`

The create-timeout artifact demonstrates idempotency. The first child entered UNKNOWN; a subsequent run before reconciliation did not generate a new client order ID. Exact lookup found the original order and moved exposure back to live open. The final reason was `CREATE_TIMEOUT_RECONCILED`.

These examples also explain why completion rate below 100 percent can be correct. If price bounds, deadline policy, minimum quantity, post-only rejection, or venue state prevent a legal completion, the system should return actual filled and unfilled quantity with a clear reason.

10 Real Failure Case and Hardening

The most important failure case found during development was ambiguous Binance create timeout handling. A timeout from `POST /fapi/v1/order` does not prove the order failed. The exchange may have accepted it while the client lost the response. If the client clears exposure and retries with a fresh client order ID, both the original and replacement can fill.

The fix was to classify create HTTP 408 and transport timeout as unknown mutation outcomes. The child stays in UNKNOWN; its quantity remains in `unknown_order_quantity`; the engine blocks new child creation that would violate the invariant; and reconciliation queries the exact `origClientOrderId`. If the order is found, it is promoted back to live open or filled state. If exact lookup proves it does not exist, unknown exposure is cleared. Broad prefix scans are useful diagnostics, but they are not proof that a specific timed-out order does not exist.

External-review hardening also improved several related paths:

- Partial fills returned in create responses now update parent fills immediately.
- Lower or duplicate cumulative snapshots cannot reduce or double-count parent fills.
- Stale market data pauses execution with a controlled reason instead of raising unexpectedly.
- Runtime unknown-reconciliation failures are recorded and retried.
- User stream events and disconnects trigger bounded execution-scoped reconciliation.
- Shutdown stops new child creation, cancels active orders, reconciles, and leaves no background execution task running.

This failure case is useful for live defense because it maps directly to a direct-deduction item in the brief and shows that the implementation was hardened against a concrete venue ambiguity, not just a theoretical edge case.

11 Limitations and Future Work

The implementation is intentionally compact. It uses in-memory state, so process restart loses execution records. It targets BTCUSDT and one-way mode. Hedge mode is rejected rather than fully supported. Runtime stream supervision is sufficient for this take-home but is not a production operations system with durable replay, leader election, alerting, or dashboards. Mainnet is configuration-compatible but hard disabled by default and should not be demonstrated as an automated trading system.

Accepted sanitized Binance Testnet evidence is included for both Chase and TWAP, but future Testnet reruns remain externally dependent on account funding, permissions, and Binance risk configuration. The older raw credentialed rejection artifacts under `reports/evidence/testnet/chase/exec_85051310eb714ebe` and `reports/evidence/testnet/twap/exec_30ed2b4cac4346a1` are retained only as rejected connectivity/error artifacts. Simulator artifacts should not be substituted for accepted Testnet artifacts.

Future work should prioritize durable event storage and replay, restart recovery, broader symbol support, richer operational monitoring, and formalizing a production runbook. Hedge-mode support could be added, but it should not be mixed into the current one-way invariant without a separate design pass because position-side semantics change the target-position calculation.

12 AI Usage

AI assistance was used for planning, code review, test design, documentation drafting, and report preparation. Suggestions were treated as drafts. The implementation was checked against the project brief, corrected where unsafe, and verified with automated tests plus deterministic simulator scripts. The main reviewed areas were create-timeout handling, unknown exposure, cancel/fill race safety, cumulative fill monotonicity, duplicate event handling, Decimal-only price and quantity paths, TWAP carry-forward, and documentation alignment.

The candidate remains responsible for explaining and modifying the submitted code. No secrets, credentials, request signatures, listen keys, or private Binance account data are included in the repository.

A Edge-Case Matrix

Table 5: Brief edge cases and implemented behavior.

ID	Edge case	Implemented behavior
E1	Target already reached	No child order; terminal no-action/completed behavior.
E2	Target below minimum order size	Reject or record dust explicitly; never silently round up into overfill.
E3	Partial fill then cancel/reprice	Replacement quantity is based on parent confirmed fills and reserved exposure.
E4	Fill during pending cancel	Pending-cancel exposure remains reserved until terminal state or reconciliation.
E5	Create request timeout	Child becomes UNKNOWN; exact client-order lookup before clearing exposure.
E6	Duplicate or out-of-order events	Trade IDs and monotonic cumulative quantity prevent double counting and state regression.
E7	Price jumps outside range	Stop active chasing; deadline policy reports actual unfilled quantity.
E8	Frequent price movement	Reprice threshold and minimum interval limit cancel storm risk.
E9	Stream disconnect or stale quote	Pause new action, reconnect, reconcile, then resume or exit safely.
E10	Deadline with remainder	Cancel remainder or make bounded aggressive attempt within range; report actual result.
E11	Post-only reject	Refresh state and retry within bounded behavior; no infinite loop.
E12	Target crosses zero	Required trade is net position delta, not close/open split confusion.

B Artifact Checklist

Final submission should include source code, tests, scripts, README, AI disclosure, this report, and selected evidence artifacts. Simulator artifacts under `reports/evidence/simulation` should include request snapshots, execution logs, execution summaries, child-order CSVs, fill CSVs, timelines, and TWAP ledgers. The accepted sanitized Testnet artifacts under `reports/evidence/testnet/chase/exec_3168600ee25b4193` and `reports/evidence/testnet/twap/exec_85bef3985ea3431a` additionally include `symbol_rules.json`, `reconciliation_orders.csv`, and `evidence_manifest.json` with non-empty exchange order IDs and accepted exchange-order evidence. The older insufficient-margin artifacts remain only rejected connectivity/error evidence.

Artifacts must not include API secrets, listen keys, request signatures, signed payloads, or private account data not needed for review. Accepted Testnet logs redact `ACCOUNT_UPDATE` balance and position fields while preserving order/trade updates needed to prove exchange acceptance.

References

George Kour. arxiv-style: A latex style and template for paper preprints. `https://github.com/kourgeorge/arxiv-style`, 2026. Repository accessed for local report template.

Calais. Calais execution algorithm candidate project brief and interviewer evaluation guide, 2026. Candidate project brief, Version 2.0.

Binance. Binance usd-m futures api documentation. `https://developers.binance.com/docs/derivatives/usds-margined-futures/`, 2026. Reference for venue concepts used by the implementation.